

NCERT Solutions For Class 12 Computer Science

Chapter 1 – Exception Handling In Python

Exercises

1. “Every syntax error is an exception but every exception cannot be a syntax error.” Justify the statement.

Answer: A syntax error is a specific type of exception that occurs when the rules of Python are not followed while writing the code. However, an exception is a broader term that includes syntax errors as well as other errors like runtime errors and logical errors that arise during the execution phase. Therefore, every syntax error is an exception, but exceptions can also be caused by other reasons and not just syntax errors.

2. When are the following built-in exceptions raised? Give examples.

a) ImportError

b) IOError

c) NameError

d) ZeroDivisionError

(a) ImportError: Raised when Python cannot find the specified module.
Example: `import module` will produce `ImportError` if 'module' does not exist.

(b) IOError: Raised when an input/output operation fails, such as opening a file that does not exist.

Example: `file = open("abc.txt", "r")` if 'abc.txt' is missing.

(c) NameError: Raised when a variable or function name is not defined.

Example: `print(value)`, if 'value' was not defined earlier.

(d) ZeroDivisionError: Raised when a number is divided by zero.

Example: `print(5/0)` will raise this error.

3. What is the use of a raise statement? Write a code to accept two numbers and display the quotient. Raise an appropriate exception if the denominator is zero (0).

Answer: The raise statement is used in Python to throw an exception during program execution, intentionally raising an error when required conditions are not met.

```
numerator = float(input("Enter the numerator: "))
denominator = float(input("Enter the denominator: "))
if denominator == 0:
    raise ZeroDivisionError("Error: Denominator cannot be zero.")
else:
    quotient = numerator / denominator
    print("Quotient:", quotient)
```

4. Use assert statement in Question 3 to test the division expression in the program.

Answer:

```
numerator = float(input("Enter the numerator: "))
denominator = float(input("Enter the denominator: "))
assert denominator != 0, "Error: Denominator cannot be zero."
quotient = numerator / denominator
print("Quotient:", quotient)
```

If the user enters 0 as denominator, the program raises an AssertionError with the message.

5. Define the following:

a) Exception Handling

b) Throwing an exception

c) Catching an exception

a) Exception Handling: Writing extra code in programs to display helpful messages or instructions to the user when an exception occurs, ensuring the program doesn't terminate unexpectedly.

b) Throwing an exception: The process where an exception object is created and passed to the runtime system or exception handler when an error occurs.

c) Catching an exception: Executing a suitable handler written to deal with a specific exception when it arises during program execution.

6. Explain catching exceptions using try and except blocks.

Answer: In Python, any code that may raise an exception is placed inside a try block. If an exception is raised, the rest of the try block stops executing and the control moves to the associated except block, where handling code is written. This structure allows specific or general exceptions to be handled gracefully without crashing the program.

try:

```
# code that can raise exception
except ExceptionName:
    # code to handle the exception
```

7. Consider the code given below and fill in the blanks.

```
print (" Learning Exceptions...")
try:
    num1= int(input ("Enter the first number"))
    num2=int(input("Enter the second number"))
    quotient=(num1/num2)
    print ("Both the numbers entered were correct")
except _____: # to enter only integers
    print (" Please enter only numbers")
except _____: # Denominator should not be zero
    print(" Number 2 should not be zero")
else:
    print(" Great .. you are a good programmer")
    _____: # to be executed at the end
    print(" JOB OVER... GO GET SOME REST")
```

Answer:

First blank: ValueError

Second blank: ZeroDivisionError

Third blank: finally

```
print (" Learning Exceptions...")
try:
    num1= int(input ("Enter the first number"))
    num2=int(input("Enter the second number"))
    quotient=(num1/num2)
    print ("Both the numbers entered were correct")
except ValueError: # to enter only integers
    print (" Please enter only numbers")
except ZeroDivisionError: # Denominator should not be zero
```

```
print(" Number 2 should not be zero")
else:
    print(" Great .. you are a good programmer")
finally: # to be executed at the end
    print(" JOB OVER... GO GET SOME REST")
```

8. Write a code where you use the wrong number of arguments for a math method (e.g., `sqrt()` or `pow()`). Use exception handling to catch `ValueError`.

Answer:

```
import math

try:
    result = math.pow(2, 3, 4, 5) # pow() expects 2 args
except TypeError:
    print("TypeError occurred with math.pow()")
else:
    print("Result:", result)

try:
    result = math.sqrt(9, 2) # sqrt() expects 1 arg
except TypeError:
    print("TypeError occurred with math.sqrt()")
else:
    print("Result:", result)
```

TypeError will be raised when an incorrect number of arguments are

provided, and the exceptions are caught and displayed.

9. What is the use of finally clause? Use finally clause in the problem in Question 7.

Answer: The finally clause is used to execute statements regardless of whether an exception occurs or not in the try block. It is often used for resource cleanup.

```
print("Learning Exceptions...")
```

```
try:
```

```
    num1 = int(input("Enter the first number: "))
```

```
    num2 = int(input("Enter the second number: "))
```

```
    quotient = (num1 / num2)
```

```
    print(quotient)
```

```
    print("Both numbers entered were correct")
```

```
except ValueError:
```

```
    print("Please enter only numbers")
```

```
except ZeroDivisionError:
```

```
    print("Number 2 should not be zero")
```

```
else:
```

```
    print("Great.. you are a good programmer")
```

```
finally:
```

```
    print("JOB OVER... GO GET SOME REST")
```